DATABASE PROJECT 1

Affordable Housing Maintenance
Work-Order Database

Case 5 | CSE 5720 — Database Systems | Fall 2026

Deliverables 1–3: Client Scope, Requirements and Scenarios, ERD and Relational Design

1. Client Problem and Scope

1.1 Client and Problem

Item	Description
Client	A fictional affordable housing nonprofit that manages properties and residential units.
Problem	The nonprofit receives maintenance requests, assigns technicians, and tracks work-order progress. Managers need a reliable way to identify overdue work, monitor assignments, and recognize recurring maintenance issues across properties.
Proposed solution	A PostgreSQL relational database that stores property and unit information, coded resident records, issue categories, technicians, work orders, technician assignments, and work-status history.

1.2 Primary Users

User	Primary responsibilities
Staff	Record requests, set priorities, and update work-order status.
Technicians	Review assigned work and track outstanding jobs.
Managers	Monitor overdue work, workload distribution, and recurring issues by property and issue category.

1.3 Decisions Supported by the Database

Decision area	Question the database should help answer
Overdue maintenance	Which open, high-priority work orders are older than seven days and require attention?
Technician workload	Which technicians have at least four open work orders, and how should work be distributed?
Recurring maintenance	Which properties and units experience repeated maintenance problems, and which units have more work orders than average?

1.4 Scope and Boundaries

The project will support maintenance-request records, assignment tracking, status history, reporting, and data-integrity checks. It will use PostgreSQL and synthetic test data only.

The project does not include:

- Building a website or mobile application
- Processing payments
- Storing real resident personal information
- Implementing a full property-management system

1.5 Initial Assumptions

- Each unit belongs to one property.
- Each work order concerns one unit, one coded resident record, and one issue category.
- A work order may have no technician assigned yet.
- Technician assignments and status changes must be recorded in the database.
- Priority and status values will be restricted to chosen values.

2. Requirements and Scenarios

2.1 Business Rules

ID	Business rule
BR-01	Each unit must belong to exactly one existing property.
BR-02	A unit number must be unique within its property. Different properties may use the same unit number.
BR-03	Each work order must reference one existing unit.
BR-04	Each work order must reference exactly one existing coded resident record. Actual resident personal information isn't included in the project's synthetic data.
BR-05	Each work order must reference one existing issue category.
BR-06	Each work order must have one valid priority value: LOW, MEDIUM, or HIGH.
BR-07	Each work order must have one valid current status: OPEN, IN_PROGRESS, COMPLETED, or CANCELLED.
BR-08	Every work order must have a submission timestamp. If a completion timestamp exists, it can't be earlier than the submission timestamp.
BR-09	A work order may exist without a technician assignment.
BR-10	Each work-assignment record must reference one existing work order and one existing technician.
BR-11	Each status-history entry must reference one existing work order and record a valid status and timestamp.
BR-12	Each technician assignment must record when the assignment was made so staff can review and track assignments.

2.2 Required Client Questions

Query	Question
Q1	List open high-priority work orders older than 7 days
Q2	Show each work order with its property, unit, issue category, assigned technician, and status
Q3	Count work orders by property and issue category
Q4	Show technicians assigned at least 4 open work orders
Q5	Find units with more work orders than the average unit

2.3 Client Scenarios

1. Operational Workflow: Reporting and Assigning a Maintenance Request

Actor: staff member

Trigger: A resident reports a plumbing problem in a residential unit

Data needed: Unit identifier,
property identifier through the unit,
coded resident identifier,
issue category, description,
submission timestamp,
priority,

current status

Database action: Staff creates a work-order record linked to the existing unit, coded resident, and issue category. The initial status is recorded, and an assignment record is created when a technician is assigned. The work order may remain unassigned temporarily.

Success criterion: The work order is stored with valid references, can be retrieved with its property and unit information, and can be assigned to a technician without duplicating the work-order record.

2. Decision/Reporting: Identifying Overdue High-Priority Work

Actor: Maintenance manager

Trigger: The manager reviews outstanding work to determine which requests require follow-up

Data needed: Work-order identifier,

submission timestamp,

priority,

current status,

unit,

property

Database action: A SQL query filters for work orders whose current status is OPEN, priority is HIGH, and submission timestamp is more than 7 days before the reporting time. Results are ordered by submission timestamp, from oldest to newest.

Success criterion: The manager receives a list of qualifying work orders and their property/unit details, allowing the oldest requests to be prioritized.

3. Exception/Edge Case: Invalid Work-Order Completion Time

Actor: staff member

Trigger: A staff member attempts to record a completion timestamp earlier than the work-order submission timestamp

Data needed: Work-order identifier,

submission timestamp,

proposed completion timestamp

Database action: A CHECK constraint rejects the invalid completion timestamp. The verification script uses a transaction and ROLLBACK to demonstrate the failure without keeping the test change.

Success criterion: The invalid record is rejected, existing valid data remains unchanged, and the test confirms that the timestamp rule is enforced by PostgreSQL.

3. ERD and Relational Design

3.1 Conceptual ERD

The conceptual ERD represents eight entities:

PROPERTY, UNIT, RESIDENT_CODE, ISSUE_CATEGORY, TECHNICIAN, WORK_ORDER, WORK_ASSIGNMENT, WORK_STATUS_HISTORY

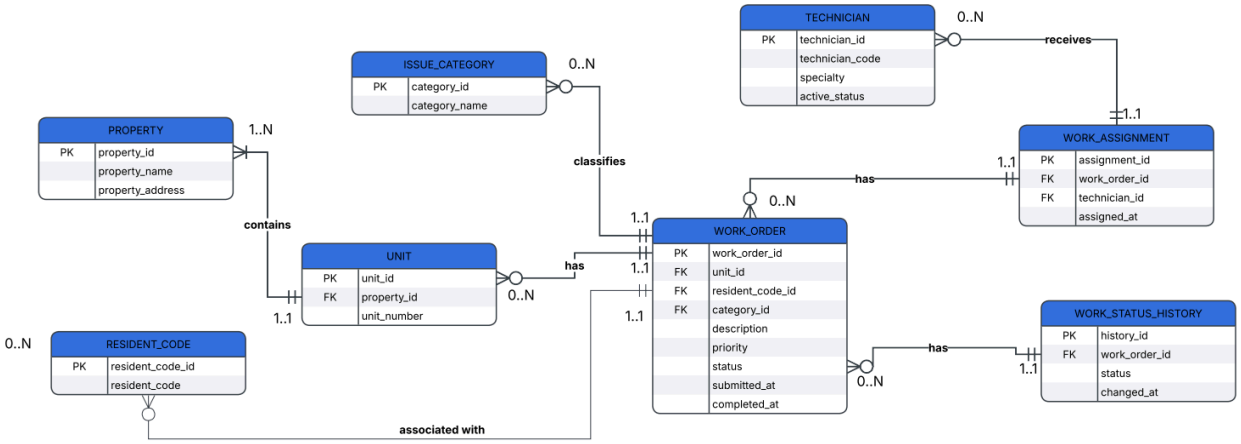


Figure 1. Conceptual ERD

3.2 Relationship Summary

Relationship	Cardinality and meaning
PROPERTY → UNIT	Each property can contain zero or many units while each unit belongs to exactly one property. Unit numbers must be unique within a property, although different properties may use the same unit number.
UNIT → WORK_ORDER	Each unit may have zero or many work orders while each work order concerns exactly one unit
RESIDENT_CODE → WORK_ORDER	Each resident code can be associated with zero or many work orders while each work order references exactly one coded resident record
ISSUE_CATEGORY → WORK_ORDER	Each issue category may classify zero or many work orders while each work order refers to exactly one existing issue category
WORK_ORDER → TECHNICIAN	Each work order may have 0 or many technicians assigned to it, while each technician may receive 0 or many work orders. This M:N Relationship becomes connected via the Associative Entity WORK_ASSIGNMNET Each work assignment references an existing work order and technician and preserves when the assignment was made thereby allowing multiple work assignments to be associated with a work order or technician
WORK_ORDER → WORK_STATUS_HISTORY	Each work order may have zero or many status-history entries where each history entry references one existing work order and records a status and timestamp, thereby preserving status changes

3.3 Relational Schema

The relational schema maps the entities and relationships to tables using primary keys, foreign keys, rules regarding uniqueness, as well as check constraints. The definitions below reflect the current proposed design; the team should confirm unresolved design choices before implementation.

PROPERTY

```
property_id          INTEGER  PRIMARY KEY
property_name        VARCHAR(100)  NOT NULL
property_address     VARCHAR(200)  NOT NULL
```

UNIT

```
unit_id          INTEGER  PRIMARY KEY
property_id      INTEGER  NOT NULL  REFERENCES PROPERTY(property_id)
unit_number      VARCHAR(20)  NOT NULL
UNIQUE (property_id, unit_number)
```

TECHNICIAN

```
technician_id       INTEGER  PRIMARY KEY
technician_code     VARCHAR(20)  NOT NULL  UNIQUE
specialty           VARCHAR(50)  NULL
active_status       BOOLEAN  NOT NULL
```

RESIDENT_CODE

```
resident_code_id  INTEGER  PRIMARY KEY
resident_code     VARCHAR(20)  NOT NULL  UNIQUE
```

ISSUE_CATEGORY

```
category_id       INTEGER  PRIMARY KEY
category_name     VARCHAR(50)  NOT NULL  UNIQUE
```

WORK_ORDER

```
work_order_id     INTEGER  PRIMARY KEY
unit_id           INTEGER  NOT NULL  REFERENCES UNIT(unit_id)
resident_code_id  INTEGER  NOT NULL  REFERENCES RESIDENT_CODE(resident_code_id)
category_id       INTEGER  NOT NULL  REFERENCES ISSUE_CATEGORY(category_id)
description       TEXT  NULL
priority          VARCHAR(10)  NOT NULL  CHECK (priority IN ('LOW', 'MEDIUM', 'HIGH'))
status        VARCHAR(20)  NOT NULL  CHECK (status IN ('OPEN', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'))
submitted_at      TIMESTAMP NOT NULL
completed_at      TIMESTAMP NULL CHECK (completed_at IS NULL OR completed_at >= submitted_at)
```

WORK_ASSIGNMENT

```
assignment_id  INTEGER  PRIMARY KEY
work_order_id  INTEGER  NOT NULL  REFERENCES WORK_ORDER(work_order_id)
technician_id     INTEGER  NOT NULL  REFERENCES TECHNICIAN(technician_id)
assigned_at     TIMESTAMP  NOT NULL
```

WORK_STATUS_HISTORY

```
history_id       INTEGER  PRIMARY KEY
work_order_id  INTEGER  NOT NULL  REFERENCES WORK_ORDER(work_order_id)
status        VARCHAR(20)  NOT NULL  CHECK (status IN ('OPEN', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'))
changed_at     TIMESTAMP  NOT NULL
```

3.4 Data Dictionary

Entity	Attribute	Data type	Key / constraint	Nullable
PROPERTY	property_id	INTEGER	PK	No
	property_name	VARCHAR(100)	—	No
	property_address	VARCHAR(200)	—	No
UNIT	unit_id	INTEGER	PK	No
	property_id	INTEGER	FK → PROPERTY(property_id)	No
	unit_number	VARCHAR(20)	UNIQUE with property_id	No
TECHNICIAN	technician_id	INTEGER	PK	No
	technician_code	VARCHAR(20)	UNIQUE	No
	specialty	VARCHAR(50)	—	Yes
	active_status	BOOLEAN	—	No
RESIDENT_CODE	resident_code_id	INTEGER	PK	No
	resident_code	VARCHAR(20)	UNIQUE	No
ISSUE_CATEGORY	category_id	INTEGER	PK	No
	category_name	VARCHAR(50)	UNIQUE	No
WORK_ORDER	work_order_id	INTEGER	PK	No
	unit_id	INTEGER	FK → UNIT(unit_id)	No
	resident_code_id	INTEGER	FK → RESIDENT_CODE(resident_code_id)	No
	Category_id	INTEGER	FK → ISSUE_CATEGORY(category_id)	No
	description	TEXT	—	Yes
	priority	VARCHAR(10)	CHECK: LOW, MEDIUM, HIGH	No
	Status	VARCHAR(20)	CHECK: OPEN, IN_PROGRESS, COMPLETED, CANCELLED	No
	submitted_at	TIMESTAMP	—	No
	completed_at	TIMESTAMP	CHECK: NULL or >= submitted_at	Yes
WORK_ASSIGNMENT	assignment_id	INTEGER	PK	No
	work_order_id	INTEGER	FK → WORK_ORDER(work_order_id)	No
	technician_id	INTEGER	FK → TECHNICIAN(technician_id)	No
	assigned_at	TIMESTAMP	—	No
WORK_STATUS_HISTORY	history_id	INTEGER	PK	No
	work_order_id	INTEGER	FK → WORK_ORDER(work_order_id)	No
	Status	VARCHAR(20)	CHECK: OPEN, IN_PROGRESS, COMPLETED, CANCELLED	No
	changed_at	TIMESTAMP	—	No

Hey guys, I reorganized and cleaned up Deliverables 1–3 so the client scope, business rules, scenarios, ERD, relational schema, and data dictionary are easier to review.

I'll also continue working on revising Deliverable 6 (the README) as much as I can in the meantime. Once the four SQL files are ready, I'll be able to review the actual implementation and update both Google Docs accordingly, including making sure the schema and data descriptions align with the first two SQL files, and that the query explanations, setup instructions, expected outputs, and verification results align with the last two.

That way, we can keep the documentation consistent with what we're actually building.

3.5 Proposed Integrity Rules and Notes

- Foreign keys must reference existing records in their designated parent tables.
- ON DELETE RESTRICT i propose we use for foreign keys in order to preserve referenced records and maintenance history
- Chosen Priority values are restricted to LOW, MEDIUM, and HIGH.
- Chosen Status values are restricted to OPEN, IN_PROGRESS, COMPLETED, and CANCELLED.
- The proposed timestamp rule is completed_at IS NULL OR completed_at >= submitted_at.

-

NOTE:These are open design questions, not finalized. align the ERD, schema, seed data, verification tests, and README before submission.

- Completion timestamp: Should a work order with status COMPLETED be required to have a non-NULL completed_at, or is the current rule (if present, it cannot be earlier than submitted_at) sufficient?
- Technician assignments: Does WORK_ASSIGNMENT handle assignment history/reassignment, or can several technicians be assigned simultaneously to one work order? The current schema permits multiple assignment rows but doesn't define which interpretation
- Status history: The current design stores both WORK_ORDER.status and WORK_STATUS_HISTORY.status. We should define how the current status and history rows remain consistent when a status changes.